

CODEX x SGLANG / ASCEND - DESK RULES

Optimize task boundaries, not just prompt length
NEW OBJECTIVE = NEW SESSION

1 Is this a new objective?

YES -> NEW SESSION

2 Is it only a Git action?

YES -> TERMINAL, not Codex

3 Need findings from another session?

HANDOFF FILE -> NEW SESSION

4 One long evidence-driven experiment?

/PLAN -> /GOAL + ARTIFACT LEDGER

A. PICK THE MODEL

Luna / low - descriptions, metadata, docs, mechanical edits

Terra / medium - conflicts, comments, normal bugs/refactors

Sol / high - deep review, hard feature, Ascend correctness/perf

Sol / xhigh - only after focused investigation fails

cxl | cx | cxh | cxx

B. BEFORE CODEX EDITS

- GOAL - what exactly should change?
- SCOPE - which code/path is allowed?
- STARTING EVIDENCE - PR, commit, error, file, benchmark?
- DO NOT CHANGE - backends, behavior, unrelated code?
- DONE - what proves the task is complete?

One concrete objective + one stop condition.

C. SEARCH - SMALLEST TOOL FIRST

Exact symbol / path / error rg

Known PR / commit / regression boundary git diff / show / bounded log

Model evolution / old SGLang optimization model-history skill

"Where is this behavior implemented?" Semble

Callers / references / symbol graph Serena only if needed

Huge log reduce first; send errors + context + metrics

D. ASCEND / NPU - KNOW THE ENVIRONMENT FIRST

Record before diagnosis / benchmark:

Hardware | CANN | torch | torch_npu | sgl-kernel-npu | SGLang commit

graph/eager | dtype/quant | TP/DP/EP | workload | actual backend path

torch_npu API / graph / streams
-> torch_npu

Analyze profile
-> profiling analysis

CUDA -> NPU review
-> adapter reviewer

HCCL
-> hccl-test

STOP benchmark if anything except the tested variable changed or fallback occurred.

Collect profile
-> profiling collection

Single op/kernel speed
-> op benchmark

Odd profiling result
-> profiling anomaly

Triton / AscendC / op-plugin
-> load only for that task

E. DO NOT WASTE A MODEL TURN

- git push / status / fetch / switch when no reasoning is needed
- standalone "push", "continue", or "update" in a long thread
- review + cleanup + comments + docs in one task
- routine Sol/xhigh "just in case"
- full huge logs before reducing them
- unrelated refactor after the requested objective is done
- tests by default - only explicit request, CI/review requirement, or real regression guard
- CUDA/NCCL assumptions copied mechanically to NPU/HCCL

F. FINISH AND STOP

- ☐ Objective complete
- ☐ Diff minimal
- ☐ Smallest useful validation done
- ☐ Needed state saved to handoff/Goal
- ☐ No reason to keep this transcript alive

CHAT = WORKING MEMORY | GIT = CODE STATE | HANDOFF / GOAL FILES = DURABLE MEMORY

Review -> handoff -> new session | Performance loop -> Goal + artifacts + compact | Faster kernel != E2E win